

There are 6 Steps in the computational Pipeline:

- 1.) Vector Embedding Tensor Extraction
(computed & precomputed)
- 2.) L2 Normalization
- 3.) Symmetric Scalar Quantization
- 4.) Binary conversion + Bitplane Disaggregation
(per row)
- 5.) Bitpacking & Concatenation of bitplanes
- 6.) ZSTD & LZ4 compression + Analysis

This guide will show the general pipeline, not the more detailed implementation information.

Step 1: Vector Embedding Tensor Extraction

If precomputed: (Accepts: .parquet, .npz, .csv)

- 1.) Autodetects column with vector embeddings
- 2.) Extracts embedding column
- 3.) Sends to next Step (normalization)

If computed: (Accepts: .parquet, .csv, .txt, .png, .jpg, .mp4, .mov, .mp3, .wav)

- 1.) Loads in file data
- 2.) (if text or parquet) User chooses column
- 3.) Data is loaded into library
- 4.) Embedding model is loaded
- 5.) Data is loaded into embedding model with config and chunking (CPU/GPU acceleration)

6a.) If text:

- ↳ each line becomes a vector embedding
- ↳ lines that exceed token context are broken down
- ↳ follows embedding model token context & pooling method

6b.) If multimodal:

- ↳ 1 vector embedding per file (by default)

Step 2: L2 Normalization

L^2 Norm/Euclidean norm is applied to each vector embedding row.

For vector embedding $R_n = \{X_1, X_2, \dots, X_d\}$

Where:

↳ R_n = Vector Embedding Row (Row # n)

↳ $X_{\#}$ = float element

↳ d = #dimensions of vector embedding (determined by model)

The L^2 Norm is applied as such:

$$\overbrace{\|R_n\|}^{L^2 \text{ normalized}} = \sqrt{X_1^2 + X_2^2 + \dots + X_d^2}$$

This results in a data range that is always between -1.000 and $+1.000$

This should result in a norm of 1.000 ± 0.001

Step 3: Symmetric Scalar Quantization

Here are the steps of Symmetric Scalar Quantization:

1.) Get the max absolute value of the data

$$\text{maxabs} = \text{MAX}(|(x_1, x_2, \dots, x_d)|)$$

2.) Calculate Scale factor, dividing maxabs by max integer representation of quantized datatype.

ex: Quant dtype: INT8

$$\text{Scale factor} = \frac{\text{maxabs}}{127}$$

max int rep of INT8

3.) Each float value is converted to an Int by dividing by Scale factor and rounding

$$Q = \text{clamp}(\text{round}(\frac{x_{\#}}{S}), -127, 127)$$

int float Scale factor min/max int rep

dequant: $x_{\#} = Q_{\#} \times S$

Step 4: Binary conversion + Bitplane Disaggregation (per row)
These steps are only applied to unquantized and quantized tensors because scalar tensors are too small

EX: Tensor = $\begin{bmatrix} [4, 8, 1, 15, 3, 12, 7, 5] \\ [2, 9, 6, 13, 10, 14, 11, 0] \end{bmatrix}$

Note: in real simulation
this would be 8 bits
Binary conversion

$$\begin{bmatrix} [0100, 1000, 0001, 1111, 0011, 1100, 0111, 0101] \\ [0010, 1001, 0110, 1101, 1010, 1110, 1011, 0000] \end{bmatrix}$$

Bitplane disaggregation: per row,
each position gets its own bitplane

$$\begin{bmatrix} [0, 1, 0, 1, 0, 1, 0, 0] \\ [1, 0, 0, 1, 0, 1, 1, 1] \\ [0, 0, 0, 1, 1, 0, 1, 0] \\ [0, 0, 1, 1, 1, 0, 1, 1] \\ \vdots \end{bmatrix}$$

next stage

Step 5: Bitpacking + concatenation

↳ For each bitplane, every 8 bits are packed into 1 byte

for example: $[1, 0, 1, 0, 1, 1, 1, \dots]$

↓ group 8 bits

$[1010111, \dots]$

↓ convert to byte

$\{0xAF, \dots\}$

LZ4 and ZSTD are Byte-level compressors, hence need to be packed into bytes

↳ due to low context, we will also concatenate all bitpacked bitplanes into a single byte stream (per vector embedding) to give more context for ZSTD and LZ4 compressors

↳ ex (from previous page)

vector embedding row

$\begin{bmatrix} [0, 1, 0, 1, 0, 1, 0, 0] \\ [1, 0, 0, 1, 0, 1, 1, 1] \\ [0, 0, 0, 1, 1, 0, 1, 0] \\ [0, 0, 1, 1, 1, 0, 1, 1] \end{bmatrix}$

bitplane #2

bitplane #4

↓ bitpack

$\begin{bmatrix} [0X54] \\ [0X97] \\ [0X1A] \\ [0X3B] \end{bmatrix}$

concatenation

$[0X540X970X1A0X3B]$

next step -
↓ compressor

Step 6: ZSTD + LZ4 compressor + Metrics

For each byte stream to compress:

- ↳ 1.) byte stream is split into chunks of BLOCK_SIZE (controllable in simulation)
- ↳ 2.) each chunk is fed into the ZSTD or LZ4 compressor
- ↳ 3.) compressed sizes are counted

metrics:

- ↳ compression ratio: $R = \frac{a}{b}$, $R = \text{ratio}$
 - $a = \text{compressed size}$
 - $b = \text{original / baseline size}$
- ↳ $< 1.0 = \text{good}$
- ↳ $> 1.0 = \text{bad (negative compression)}$
- ↳ $= 1.0 = \text{neutral (no compression savings)}$

- ↳ % Space Savings:

$$\text{Savings} = \left(\left(1 - \left(\frac{a}{b} \right) \right) \times 100\% \right) \text{ rounded to 3 decimal places}$$

- ↳ $> 0\% = \text{good (space saving)}$ $a = \text{compressed size}$
- ↳ $= 0\% = \text{no effect}$
- ↳ $< 0\% = \text{bad (negative space saving)}$ $b = \text{baseline size}$